SIMULADOR MODBUS PARA TESTES DA PLATAFORMA EHUB

André Coelho Fernandes



Departamento de Engenharia Eletrotécnica

Instituto Superior de Engenharia do Porto

2025

Este relatório satisfaz, parcialmente, os requisitos que constam da Ficha de Unidade Curricular de Projeto/Estágio, do 3º ano, da Licenciatura em Engenharia Eletrotécnica e de Computadores

Candidato: André Fernandes, Nº 1221893, 1221893@isep.ipp.pt
Orientação científica: Jorge Botelho da Costa Mamede, jbm@isep.ipp.pt
Empresa: Pragmasoft - Sistemas de Automação e Controlo
Orientador: Filipe Martins Fernandes, filipe.fernandes@pragmasoft.pt



Departamento de Engenharia Eletrotécnica
Instituto Superior de Engenharia do Porto
8 de setembro de 2025

Agradecimentos

Gostaria de expressar a minha gratidão à Pragmasoft por me terem acolhido e apresentado um desafio onde tive a oportunidade de aprender muito. Em especial, gostaria de agradecer ao Engenheiro Filipe Fernandes, pelo apoio incondicional e compreensão.

Também quero agradecer profundamente ao Engenheiro Jorge Mamede pela sua orientação, por toda a ajuda prestada em busca do melhor resultado e por todo o seu otimismo.

Por fim, ao Instituto Superior de Engenharia do Porto, e a todos os colegas que fizeram parte deste percurso, deixo também aqui o meu obrigado.

Resumo

O presente relatório descreve o trabalho desenvolvido no âmbito do estágio curricular da Licenciatura em Engenharia Eletrotécnica e de Computadores, realizado na empresa Pragmasoft, no contexto do projeto Asprela + Sustentável. Este projeto visa a criação de uma comunidade de energia inteligente, baseada em produção fotovoltaica, sistemas de armazenamento e mecanismos de gestão que retirem o maior proveito do autoconsumo e da eficiência energética da microrrede instalada na zona da Agra do Amial.

No decorrer do estágio foi atribuída a responsabilidade de preparar e validar os testes à plataforma eHub, sistema central de monitorização e gestão da comunidade energética. Para alcançar este objetivo tornou-se essencial a utilização de um simulador que reproduzisse o comportamento de dispositivos compatíveis com o protocolo industrial Modbus.

Em vez de recorrer a soluções externas, foi desenvolvido de raiz um simulador Modbus em linguagem C, inspirado no funcionamento do ModRSsim2, mas adaptado às necessidades específicas do projeto. Este simulador representa o contributo mais relevante do trabalho, uma vez que permite não só editar registos manualmente como também executar scripts de teste em VBScript (.vbs), garantindo a reprodução de cenários reais de operação definidos em colaboração com a Pragmasoft.

Para suportar os ensaios, foi igualmente criada uma infraestrutura dedicada composta por duas máquinas virtuais (frontend e backend do eHub) com Ubuntu 18.04, nas quais foram restaurados os respetivos backups fornecidos pelo cliente. A base de dados MySQL, acedida via DBeaver, foi configurada de forma a assegurar a comunicação com o simulador.

Foi ainda elaborado um documento com cenários de simulação, abrangendo situações práticas como carregamento e descarga de baterias, deteção de défices energéticos, gestão de autoconsumo, alarmes de falhas de comunicação e anomalias de consumo. Cada cenário possui um script associado, pronto a ser executado pelo simulador.

A contribuição central traduz-se assim no desenvolvimento de uma ferramenta própria de simulação Modbus e na preparação de uma infraestrutura de testes robusta e reutilizável, assegurando um impacto positivo no avanço do projeto e no sucesso da plataforma eHub.

Palavras-Chave

Modbus, Simulador, eHub, Micro-rede, MySQL, VirtualBox, NetBeans, Sustentabilidade

Abstract

This report describes the work developed within the scope of the curricular internship of the Bachelor's Degree in Electrical and Computer Engineering, carried out at the company Pragmasoft, in the context of the Asprela + Sustainable project. This project aims to create an intelligent energy community, based on photovoltaic generation, storage systems, and management mechanisms that maximize self-consumption and energy efficiency in the microgrid installed in the Agra do Amial area.

During the internship, the responsibility assigned was to prepare and validate the tests for the eHub platform, the central system for monitoring and managing the energy community. To achieve this goal, it became essential to use a simulator capable of reproducing the behavior of devices compatible with the industrial Modbus protocol.

Instead of relying on external solutions, a Modbus simulator in C language was developed from scratch, inspired by the functionality of ModRSsim2, but adapted to the specific needs of the project. This simulator represents the most relevant contribution of the work, as it allows not only the manual editing of registers but also the execution of test scripts in VBScript (.vbs), ensuring the reproduction of real operating scenarios defined in collaboration with Pragmasoft.

To support the tests, a dedicated infrastructure was also created, consisting of two virtual machines (frontend and backend of the eHub) running Ubuntu 18.04, in which the respective backups provided by the client were restored. The MySQL database, accessed via DBeaver, was configured to ensure communication with the simulator.

A simulation scenarios document was also prepared, covering practical situations such as battery charging and discharging, detection of energy deficits, self-consumption management, communication failure alarms, and consumption anomalies. Each scenario includes an associated script, ready to be executed by the simulator.

The central contribution is therefore the development of a proprietary Modbus simulation tool and the preparation of a robust and reusable testing infrastructure, ensuring a positive impact on the progress of the project and on the success of the eHub platform.

Keywords

Modbus, Simulator, eHub, Microgrid, MySQL, VirtualBox, NetBeans, Sustainability

Índice

AGRADECIMENTOS ..... **I**

RESUMO ..... **III**

ABSTRACT ..... **V**

ÍNDICE ..... **VII**

ÍNDICE DE FIGURAS ..... **IX**

ÍNDICE DE LISTAGENS ..... **X**

ÍNDICE DE TABELAS ..... **XI**

ACRÓNIMOS **13**

1. INTRODUÇÃO **14**

1.1. CONTEXTUALIZAÇÃO 14
1.2. OBJETIVOS 15
1.3. CALENDARIZAÇÃO 15
1.4. ORGANIZAÇÃO DO RELATÓRIO 16
1.5. ARQUITETURA GERAL DA INFRAESTRUTURA DE TESTES 17

2. ESTADO DA ARTE **18**

2.1. PROTOCOLO MODBUS 18
2.2. SIMULADORES MODBUS 23
2.3. PLATAFORMAS DE GESTÃO DE MICRORREDES 24

3. FERRAMENTAS E TECNOLOGIA **26**

3.1. AMBIENTE DE DESENVOLVIMENTO 26
3.2. LINGUAGENS DE PROGRAMAÇÃO E SCRIPTS 28
3.3. BASE DE DADOS 29

4. DESENVOLVIMENTO **30**

4.1. CONSTRUÇÃO DO SIMULADOR 30
4.2. GESTÃO DOS REGISTOS MODBUS 31
4.3. GESTÃO DE SOCKETS 32
4.4. CONSOLA DE INTERAÇÃO 33
4.5. MOTOR DE SCRIPTS 35
4.6. FUNÇÃO PRINCIPAL 36

5. RESULTADOS **38**

5.1. ESTADO ATUAL DA INFRAESTRUTURA 38

5.2. Cenários de teste definidos 38
5.3. Cenários de teste definidos 39
5.4. Avaliação Qualitativa e Resultados Esperados 41

6. CONCLUSÕES 43

6.1. Discussão e conclusão 43
6.2. Fatores limitadores 43
6.3. Melhorias para trabalho futuro 44

REFERÊNCIAS DOCUMENTAIS 45

ANEXO A. CENÁRIOS DE SIMULAÇÃO 47

ANEXO B. CÓDIGO-FONTE DO SIMULADOR 49

HISTÓRICO ..... ERRO! MARCADOR NÃO DEFINIDO.

Índice de Figuras

Figura 1- Arquitetura geral 17
Figura 2-Arquitetura de comunicação Modbus TCP [5] 19
Figura 3- Estrutura de uma mensagem Modbus TCP [6] 20
Figura 4- Interface ModbusTcpTool 24
Figura 5- Interface ModRSsim2 24
Figura 6- Arquitetura da microrrede do projeto Asprela + Sustentável [11] 25
Figura 7- VM frontend 26
Figura 8- Vm backend 27
Figura 9- Código fonte no VSCode 28
Figura 10- Exemplo VBScript 29
Figura 11- Base de dados MYSQL no Dbeaver 29
Figura 12- Arquitetura conceptual do simulador Modbus TCP/IP 31
Figura 13- Fluxograma modbus_utils.c 32
Figura 14- Fluxograma server_utils.c 33
Figura 15- Fluxograma consola.c 34
Figura 16- Consola do Simulador 35
Figura 17- Fluxograma run_vbs.c 36
Figura 18- Fluxograma main.c 37
Figura 19- Exemplo de alteração manual de um registo (SET) e verificação posterior através de leitura (GET) no simulador Modbus. 40
Figura 20- Execução de script em VBScript no simulador 41

Índice de listagens

Listagem A. 1- Script em VBScript para carregamento por produção elevada e SOC baixo. 47
Listagem A. 2- Script em VBScript para preparação energética por atividade agendada 47
Listagem A. 3- Script em VBScript para Produção zero em horário solar 48
Listagem A. 4- main.c 50
Listagem A. 5- modbus_utils.c 51
Listagem A. 6- server_utils.c 52
Listagem A. 7- consola.c 53
Listagem A. 8- run_vbs.c 55
Listagem A. 9- user_interface.c 56

Índice de Tabelas

Tabela 1- Calendarização do projeto 16
Tabela 2- Exemplo de pedido Modbus TCP 21
Tabela 3- comparação com outros protocolos de comunicação utilizados em automação e energia. 22
Tabela 4- Exemplos de cenários de teste definidos 39

Acrónimos

LEEC	–	Licenciatura em Engenharia Eletrotécnica e de Computadores
VM	–	Virtual Machine
IP	–	Internet Protocol
SQL	–	Structured Query Language
MySQL	–	My Structured Query Language
TCP	–	Transmission Control Protocol
MBAP	–	Modbus Application Protocol Header
HR	–	Holding Register
SOC	–	*State of Charge* (Estado de Carga da bateria)
GUI	–	*Graphical User Interface* (Interface Gráfica)
VBS	–	Visual Basic Script
UPAC	–	Unidade de Produção para Autoconsumo

1. INTRODUÇÃO

O presente relatório surge no âmbito do estágio curricular da Licenciatura em Engenharia Eletrotécnica e de Computadores (LEEC) do Instituto Superior de Engenharia do Porto (ISEP), realizado na empresa Pragmasoft entre 28 de abril de 2025 e 8 de setembro de 2025, sob a orientação científica do Professor Jorge Mamede e a orientação empresarial do Engenheiro Filipe Fernandes.

1.1. CONTEXTUALIZAÇÃO

O estágio enquadrou-se no projeto Asprela+ Sustentável, que visa a criação de uma comunidade de energia inteligente na zona da Agra do Amial, com recurso a produção fotovoltaica, sistemas de armazenamento e mecanismos de gestão de autoconsumo. A plataforma eHub assume um papel central neste contexto, uma vez que é responsável pela monitorização e controlo dos diferentes ativos da microrrede, integrando dispositivos compatíveis com o protocolo industrial Modbus, assegurando a recolha de dados em tempo real, o controlo remoto e a gestão de alarmes.

Do ponto de vista técnico, o eHub implementa estratégias de gestão de autoconsumo, recorrendo a sistemas de armazenamento para absorver excedentes de produção e compensar défices de consumo.

Esta abordagem permite maximizar a utilização local da energia renovável, reduzir a dependência da rede pública e garantir a operação eficiente da comunidade energética.

Foi neste enquadramento que surgiu a necessidade de desenvolver uma solução capaz de testar de forma exaustiva a plataforma eHub. Para tal, era fundamental dispor de um simulador que reproduzisse o comportamento de dispositivos Modbus, permitindo validar a resposta da plataforma a diferentes cenários operacionais.

1.2. OBJETIVOS

O principal objetivo do estágio consistiu em preparar e validar a execução de testes à plataforma eHub, assegurando que esta responde corretamente a condições de operação realistas da microrrede. Para cumprir este propósito, foram definidos os seguintes objetivos específicos:

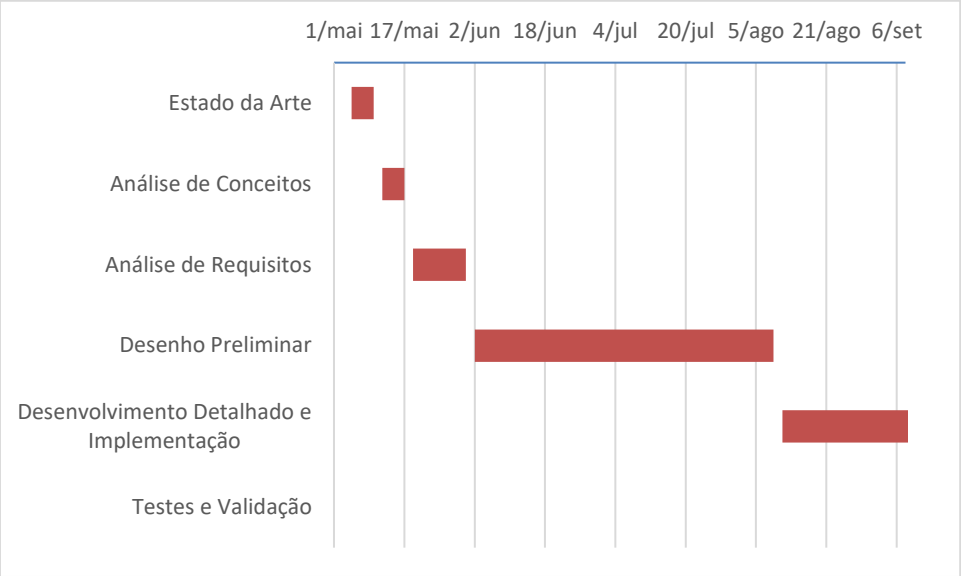
- Desenvolver um simulador Modbus em linguagem C, inspirado no funcionamento do ModRSsim2, mas adaptado às necessidades do projeto;
- Criar uma infraestrutura de testes com máquinas virtuais dedicadas (frontend e backend do eHub), incluindo a configuração da base de dados MySQL e a respetiva integração com o simulador;
- Elaborar um conjunto de cenários de simulação, recorrendo a scripts em VBScript (.vbs), que permitissem avaliar a plataforma em diferentes condições de produção, consumo, armazenamento e falhas técnicas;
- Garantir que a solução desenvolvida pode ser reutilizada pela empresa em futuros testes, constituindo uma ferramenta autónoma e de valor acrescentado para o projeto Asprela+ Sustentável.

1.3. CALENDARIZAÇÃO

Este projeto foi desenvolvido durante 19 semanas, seguindo o planeamento representado na Tabela 1.1. O trabalho teve início com o enquadramento do tema e pela elaboração de uma breve introdução ao estado da arte. De seguida, realizou-se a análise de conceitos e requisitos necessários ao desenvolvimento do simulador e à preparação da infraestrutura de testes.

Numa fase posterior, foi elaborado o desenho preliminar da solução, que serviu de base à etapa de desenvolvimento detalhado e implementação, onde foram aplicadas as ferramentas e tecnologias selecionadas. Por fim, procedeu-se à preparação dos testes e validação, através da definição de um conjunto de cenários de simulação, assegurando as condições necessárias para a execução futura dos ensaios à plataforma eHub.

Tabela 1- Calendarização do projeto



1.4. ORGANIZAÇÃO DO RELATÓRIO

O relatório encontra-se estruturado em seis capítulos. No Capítulo 1 – Introdução, é apresentado o enquadramento do projeto, os objetivos, a calendarização e a organização do documento. O Capítulo 2 – Estado da Arte reúne um enquadramento teórico, abordando o protocolo Modbus, simuladores de dispositivos e plataformas de gestão de microrredes. O Capítulo 3 – Ferramentas e Tecnologias descreve o ambiente de desenvolvimento, as bases de dados, as linguagens de programação e as ferramentas utilizadas. O Capítulo 4 – Desenvolvimento apresenta em detalhe o trabalho realizado, com especial ênfase no simulador Modbus e na infraestrutura de testes. No Capítulo 5 – Resultados, são analisados o estado atual do projeto, os cenários de simulação definidos e os contributos alcançados. Por fim, o Capítulo 6 – Conclusões e Trabalho Futuro sintetiza os principais resultados, reflete sobre o impacto do estágio e aponta sugestões para a evolução da solução desenvolvida.

1.5. ARQUITETURA GERAL DA INFRAESTRUTURA DE TESTES

Para contextualizar a infraestrutura utilizada no estágio, apresenta-se na Figura 1 a arquitetura geral do ambiente de testes. Esta arquitetura permitiu inserir o simulador Modbus TCP/IP, com a plataforma eHub, composta por duas máquinas virtuais (frontend e backend). O backend, além de disponibilizar os serviços principais da plataforma, inclui também a base de dados MySQL, à qual foi associado o DBeaver como ferramenta de administração.

A comunicação entre os diferentes componentes processa-se através de dois mecanismos principais:

- Protocolo Modbus TCP/IP, estabelecendo a ligação entre os clientes (eHub e ModbusTCPTool) e o simulador Modbus, que atua como servidor;
- Interface HTTP/REST, que assegura a interação entre o frontend (camada web) e o backend (camada de serviços e base de dados).

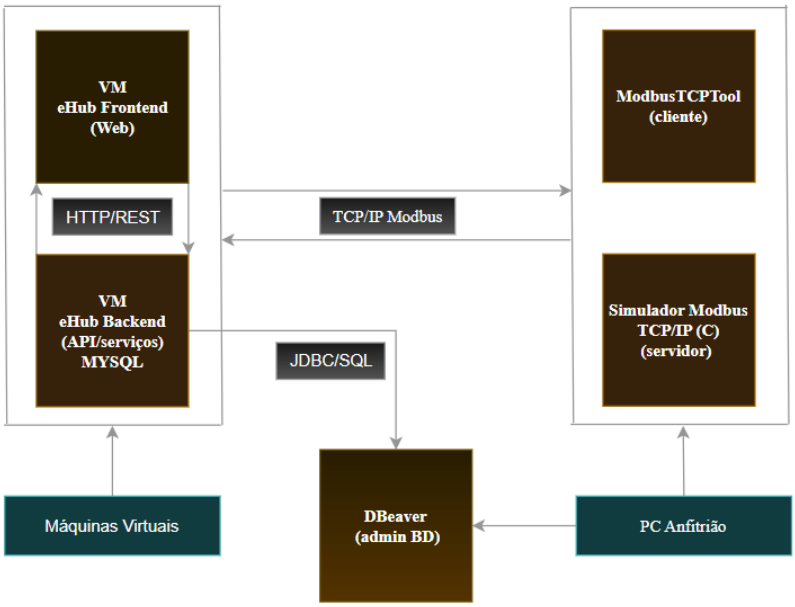


Figura 1- Arquitetura geral

2. ESTADO DA ARTE

O presente capítulo, faz uma breve introdução ao estado da arte e apresenta as ferramentas e tecnologias que permitem alcançar o objetivo a que se propõe.

2.1. PROTOCOLO MODBUS

O protocolo Modbus é um dos mais utilizados em sistemas de automação industrial, pela sua simplicidade e robustez entre equipamentos de diferentes fabricantes [1]. A sua arquitetura apresenta no modelo cliente-servidor (tradicionalmente designado como mestre-escravo), em que um dispositivo central (cliente) envia pedidos e os dispositivos remotos (servidores) respondem com os dados requisitados ou confirmam comandos [2]. Atualmente, o protocolo Modbus pode ser implementado em diferentes variantes, destacando-se o Modbus RTU (sobre interfaces como RS-232/RS-485) e o Modbus TCP (sobre redes Ethernet/IP). No projeto Asprela + Sustentável, a plataforma eHub comunicou com dispositivos através do Modbus TCP, uma vez que este garante maior flexibilidade de integração em redes locais e permite a interligação com a base de dados MySQL [3].

O protocolo organiza a informação em registos, agrupados de acordo com a sua função:

- Coils (0xxxx) – registos digitais de leitura/escrita;
- Discrete Inputs (1xxxx) – registos digitais apenas de leitura;
- Input Registers (3xxxx) – registos analógicos apenas de leitura;
- Holding Registers (4xxxx) – registos analógicos de leitura/escrita.

No âmbito deste estágio, a interação entre o simulador desenvolvido e a plataforma eHub recorreu exclusivamente à função Read Holding Registers (03), uma vez que esta permite a leitura de valores em registos analógicos. Tal abordagem foi suficiente para garantir a representação dos cenários de teste definidos, assegurando que o eHub recebia dados simulados como se se tratasse de dispositivos físicos reais [4].

A Figura 1 ilustra a arquitetura típica de comunicação Modbus TCP, evidenciando a relação entre cliente (por exemplo, eHub ou HMI) e servidor (PLC) através de rede Ethernet.

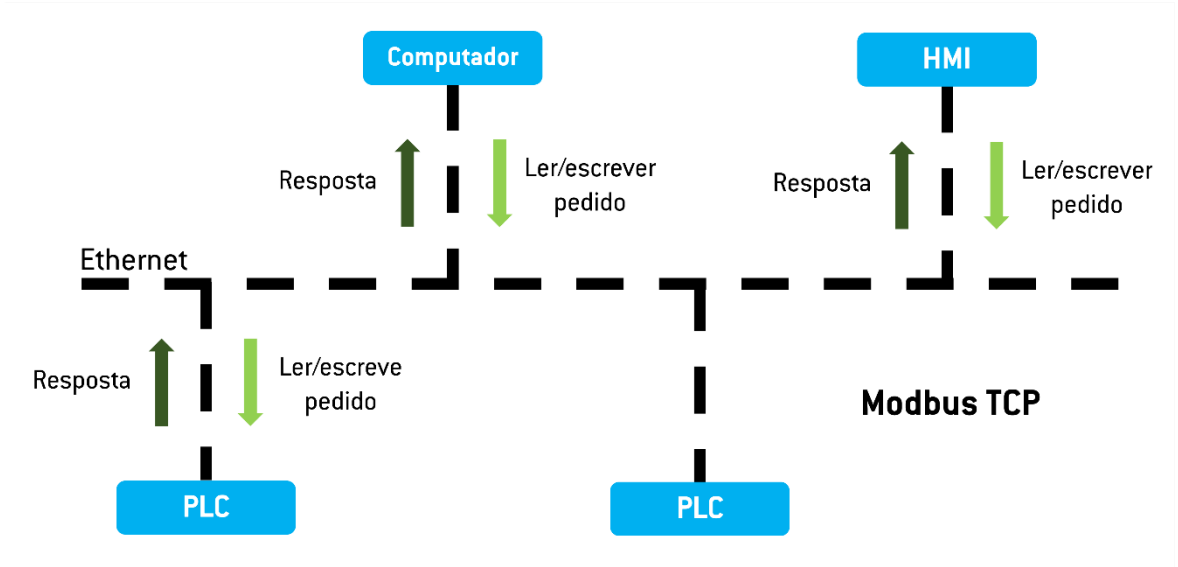


Figura 2-Arquitetura de comunicação Modbus TCP [5]

2.1.1. ORIGEM E EVOLUÇÃO

O protocolo Modbus foi inicialmente desenvolvido em 1979 pela empresa Modicon, com o objetivo de padronizar a comunicação entre controladores lógicos programáveis (PLC) e outros dispositivos industriais [1]. Desde então, tornou-se um protocolo de referência em diversos setores, nomeadamente na automação, energia e monitorização remota, pela sua facilidade de implementação e por não implicar custos de licenciamento [2].

Ao longo do tempo, a sua utilização expandiu-se para além da comunicação série (Modbus RTU), sendo posteriormente criada a versão Modbus TCP, que opera sobre redes Ethernet, potenciando a integração em sistemas distribuídos e plataformas de supervisão. Esta evolução permitiu ao protocolo manter-se relevante no contexto da Indústria 4.0, estando ainda presente em projetos atuais que envolvem gestão de energia, microrredes e comunidades de autoconsumo [3].

2.1.2. FUNCIONAMENTO DO PROTOCOLO

O protocolo Modbus baseia-se numa estrutura simples de troca de mensagens entre um cliente e um servidor. Cada comunicação é constituída por um pedido, enviado pelo cliente, e uma resposta, devolvida pelo servidor [1].

No caso do Modbus TCP, cada mensagem é encapsulada num cabeçalho TCP/IP, o que permite a sua transmissão em redes Ethernet. A estrutura típica de uma trama Modbus TCP inclui os seguintes campos [2]:

- Transaction Identifier (2 bytes) – número que permite associar pedidos e respostas;
- Protocol Identifier (2 bytes) – indica o tipo de protocolo, sendo 0 para Modbus;
- Length (2 bytes) – comprimento da mensagem;
- Unit Identifier (1 byte) – identifica o dispositivo remoto;
- Function Code (1 byte) – especifica a operação (por exemplo, leitura de registos);
- Data – contém os parâmetros do pedido ou os valores retornados.

A Figura 2 apresenta a estrutura geral de uma mensagem Modbus TCP, evidenciando os campos que compõem o cabeçalho e a área de dados.

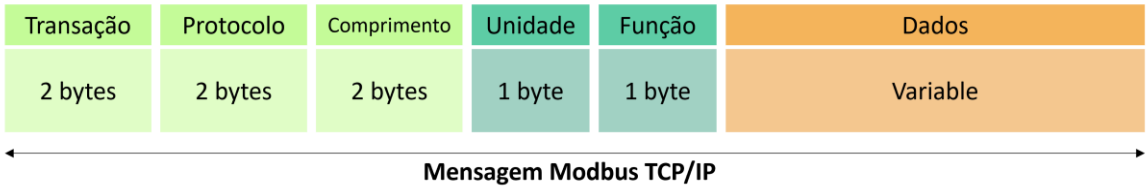


Figura 3- Estrutura de uma mensagem Modbus TCP [6]

No decorrer do estágio, a função utilizada foi a Read Holding Registers (03), que permite ao cliente solicitar ao servidor a leitura de um conjunto de registos analógicos (na gama 400001 a 499999). O servidor responde com os valores guardados nestes registos, possibilitando o controlo de variáveis como energia produzida, potência instantânea ou o estado de dispositivo [3].

A Tabela 2 exemplifica um pedido típico da função 03, onde o cliente solicita a leitura de 10 registos a partir do endereço 400001.

Tabela 2- Exemplo de pedido Modbus TCP

Campo	Tamanho (bytes)	Valor Exemplo (hex)	Descrição
Transaction ID	2	00 01	Número atribuído pelo cliente para identificar a transação
Protocol ID	2	00 00	Sempre 0x0000 em Modbus TCP
Length	2	00 06	Número de bytes a seguir (Unit ID + PDU), por exemplo 6
Unit ID	1	1	Identifica o slave (endereço Modbus)
Function Code	1	3	Código da função: 0x03 = Read Holding Registers
Starting Address	2	00 09	Endereço inicial = 9 (que corresponde ao registo 400010)
Quantity	2	00 03	Número de registos a ler = 3

2.1.3. IMPORTÂNCIA NO SETOR ENERGÉTICO

O protocolo Modbus tornou-se uma tecnologia de referência no setor energético pela sua simplicidade, baixo custo e compatibilidade com equipamentos de diferentes fabricantes [1]. É amplamente utilizado em sistemas SCADA (Supervisory Control and Data Acquisition), inversores fotovoltaicos, baterias de armazenamento e controladores programáveis (PLC), possibilitando a recolha e o controlo de dados de operação em tempo real [7].

A utilização da variante Modbus TCP facilita a integração destes dispositivos em redes Ethernet e em plataformas de gestão, como o eHub [3]. Assim, o protocolo constitui uma solução fiável para projetos de microrredes e comunidades de energia, onde a monitorização e o controlo são fatores muito importantes.

A Tabela 3 apresenta uma comparação entre o protocolo Modbus e outros protocolos de comunicação utilizados em automação e energia.

Tabela 3- comparação com outros protocolos de comunicação utilizados em automação e energia.

Protocolo	Tipo de comunicação	Vantagens principais	Limitações principais	Aplicações comuns
Modbus TCP	Cliente-servidor	Simplicidade, gratuito, amplamente suportado	Baixa segurança nativa, pouco flexível para grandes volumes	Energia, automação, SCADA
PROFINET	Ethernet industrial	Elevada velocidade, robustez, integração com PLCs	Maior complexidade, custos de licença	Automação industrial, fabrico
BACnet	Rede orientada a objetos	Normalizado, interoperabilidade em edifícios	Menos usado fora da gestão técnica	Gestão técnica de edifícios (HVAC)
DNP3	Orientado a eventos	Fiabilidade, segurança, usado em utilities	Mais complexo de implementar	Redes elétricas, subestações
IEC 61850	Orientado a serviços	Elevada escalabilidade, normalizado internacionalmente	Exige maior infraestrutura	Subestações elétricas, smart grids

2.1.4. UTILIZAÇÃO NO EHUB

No âmbito do projeto Asprela + Sustentável, o protocolo Modbus TCP é a base da comunicação entre os dispositivos da microrrede e a plataforma eHub. O eHub atua como cliente Modbus, enviando pedidos de leitura aos dispositivos ou simuladores configurados como servidores, de forma a recolher variáveis elétricas e estados de operação [3].

Entre os equipamentos com interfaces Modbus TCP incluem-se inversores fotovoltaicos, unidades de armazenamento de energia, controladores de carga e sistemas de monitorização de consumo. A padronização do protocolo permite ao eHub integrar todos estes elementos numa base de dados central, acessível para análise e tomada de decisão [1].

2.2. SIMULADORES MODBUS

2.2.1. DEFINIÇÃO E IMPORTÂNCIA DOS SIMULADORES

Os simuladores Modbus são ferramentas de software que permitem simular o comportamento de dispositivos industriais, reproduzindo a troca de dados entre cliente e servidor sem necessidade de um hardware. A sua utilização é maioritariamente em fases de desenvolvimento e teste, uma vez é possível validar a comunicação e o tratamento de dados em plataformas SCADA ou de gestão energética antes da implementação em sistemas reais [8].

Estes simuladores contribuem para reduzir custos e riscos, ao permitir a criação de cenários controlados, detetando falhas de comunicação ou erros lógicos sem que haja a possibilidade de danificar equipamentos físicos.

2.2.2. FERRAMENTAS DE APOIO UTILIZADAS

Os simuladores Modbus são amplamente utilizados em ambientes de teste, permitindo validar a comunicação entre plataformas e dispositivos sem necessidade de recorrer a hardware físico. Durante o estágio, o ModRSsim2 foi analisado e serviu de referência para o desenho do simulador desenvolvido, devido à sua simplicidade de configuração e utilização [9].

Para efeitos de validação, recorreu-se ainda ao Modbustcptool, uma ferramenta cliente Modbus TCP, utilizada unicamente para confirmar que os registos simulados eram corretamente interpretados pela plataforma eHub.

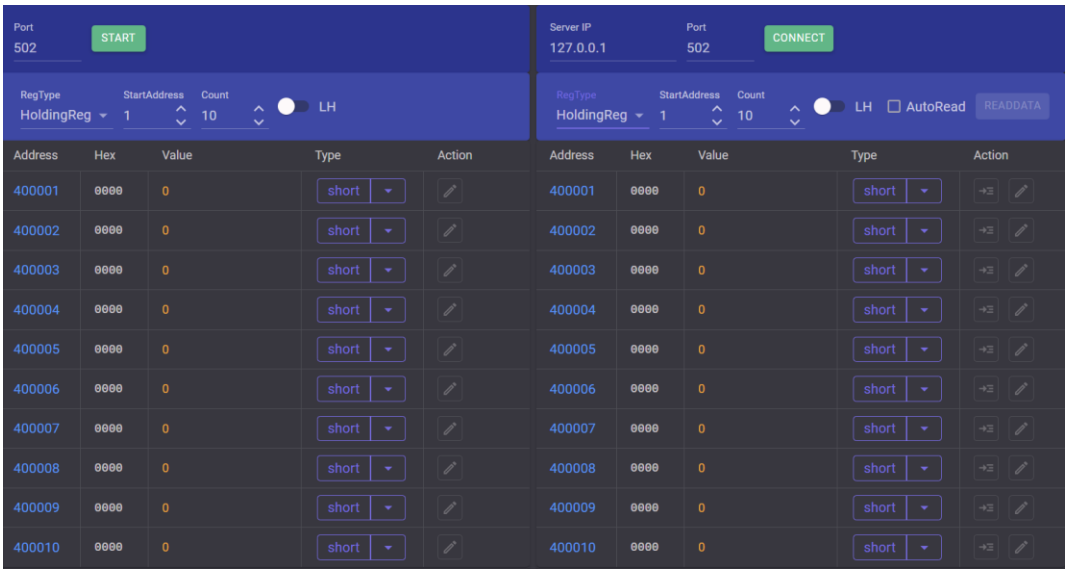


Figura 4- Interface ModbusTcpTool



Figura 5- Interface ModRSsim2

2.3. PLATAFORMAS DE GESTÃO DE MICRORREDES

As microrredes são redes elétricas de pequena escala, capazes de trabalhar de forma autónoma ou conectadas à rede elétrica principal. Tipicamente fazem parte de fontes renováveis (como sistemas fotovoltaicos), unidades de armazenamento e cargas locais, sendo manipuladas por mecanismos de controlo e otimização que trazem o equilíbrio entre produção, consumo e armazenamento [10].

Para garantir o bom funcionamento destas infraestruturas, são necessárias plataformas de gestão de microrredes, que desempenham funções de monitorização, análise e controlo. Estas plataformas recolhem dados em tempo real, identificam situações anormais e aplicam estratégias de gestão de energia que maximizam o autoconsumo, reduzem picos de carga e melhoram a funcionamento da rede [11].

No contexto do projeto Asprela + Sustentável, a plataforma eHub desempenha esse papel central. Desenvolvida para integrar diferentes equipamentos da microrrede através do protocolo Modbus TCP, o eHub recolhe variáveis como potência instantânea, energia acumulada, níveis de armazenamento, alarmes de dispositivos, entre outros. Estes dados são armazenados numa base de dados MySQL e disponibilizados ao utilizador através de uma interface gráfica acessível via internet, com as suas credenciais de entrada.

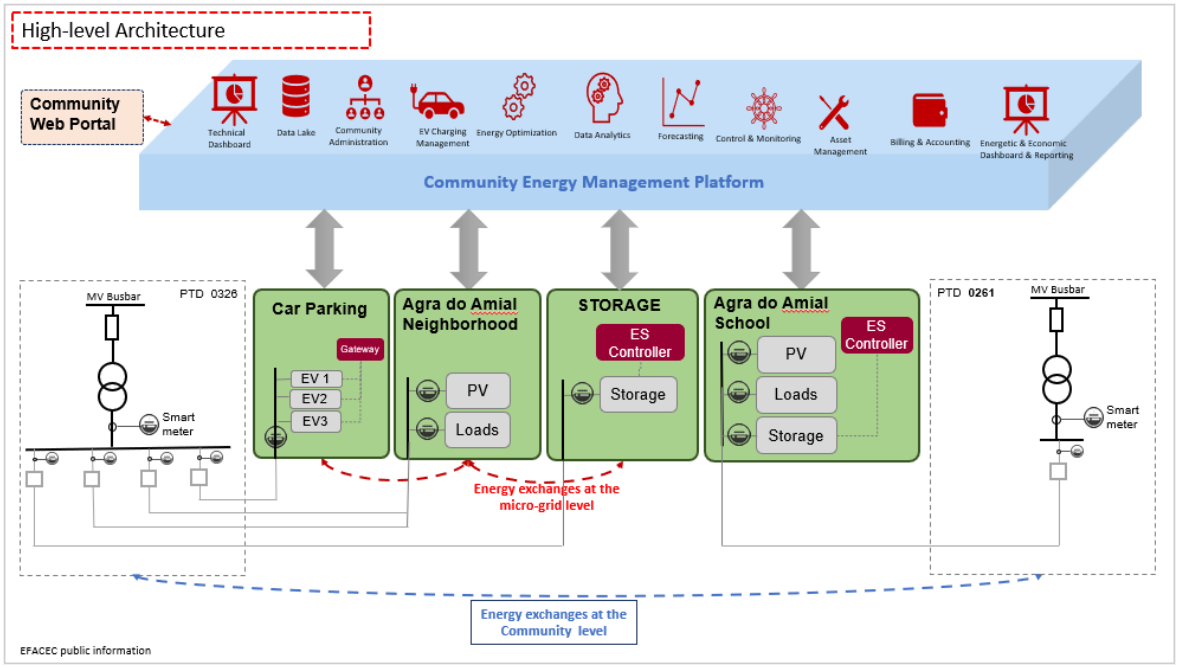


Figura 6- Arquitetura da microrrede do projeto Asprela + Sustentável [11]

3. FERRAMENTAS E TECNOLOGIA

3.1. AMBIENTE DE DESENVOLVIMENTO

3.1.1. MÁQUINAS VIRTUAIS E SISTEMA OPERATIVO

Para construir o ambiente de testes foi usado o VirtualBox, através da criação de duas máquinas virtuais independentes com Ubuntu 18.04 LTS. Cada máquina foi configurada para desempenhar funções específicas:

- Frontend da plataforma eHub;
- Backend da plataforma eHub.

Foram extraídos os backups fornecidos pela EFACEC para a raiz de cada VM, permitindo a reconstrução do ambiente de trabalho. Para assegurar a comunicação entre as máquinas virtuais e o computador anfitrião, foi necessário configurar os endereços IP e validar a ligação de rede. Esta configuração foi essencial para possibilitar que o simulador desenvolvido interagisse corretamente com a base de dados e com a aplicação.

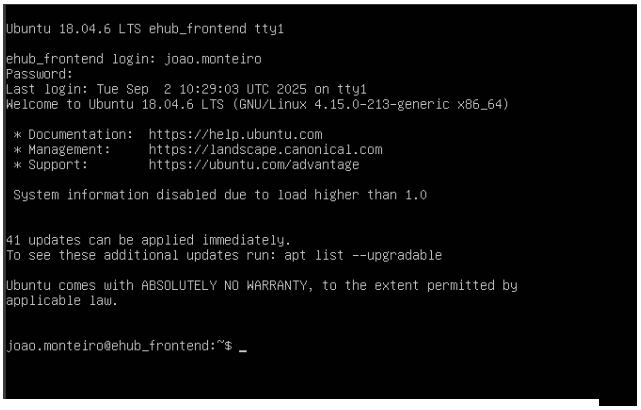


Figura 7- VM frontend

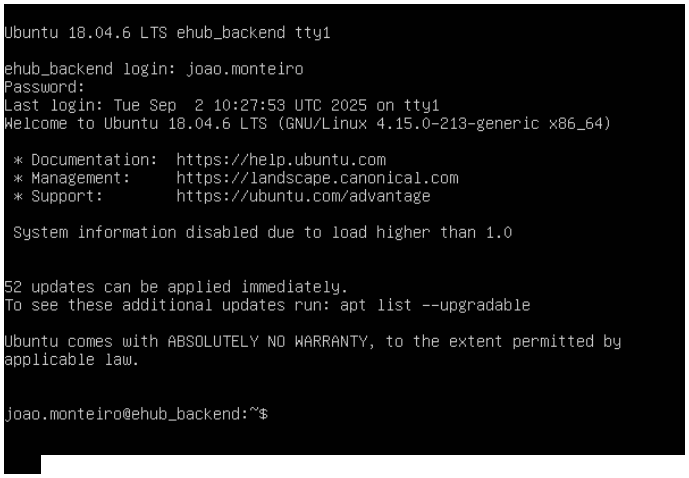


Figura 8- Vm backend

3.1.2. IDEs E COMPILADORES

Para o desenvolvimento do simulador foi utilizado o NetBeans 12.2, escolhido pela sua simplicidade e suporte a múltiplas linguagens. Foi necessário instalar a extensão de suporte à linguagem C para a edição e compilação do código.

De forma a garantir compatibilidade, recorreu-se ainda ao Cygwin, que forneceu um conjunto de bibliotecas e ferramentas de compilação. Este ambiente permitiu construir o simulador de raiz, sem recurso a bibliotecas externas para Modbus.

3.1.3. EDITORES DE TEXTO E APOIO AO DESENVOLVIMENTO

A criação dos scripts de teste (.vbs) foi realizada num editor de texto simples (Notepad), sendo os ficheiros posteriormente associados ao simulador.

O Visual Studio Code foi utilizado exclusivamente para explorar o código-fonte da plataforma eHub, permitindo compreender melhor a sua arquitetura e a forma como esta interage com dispositivos externos.

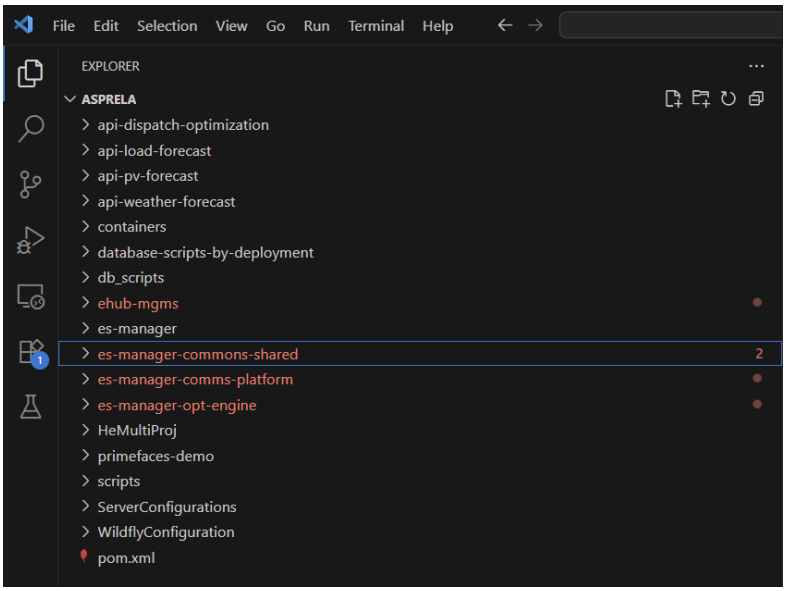


Figura 9- Código fonte no VSCode

3.2. LINGUAGENS DE PROGRAMAÇÃO E SCRIPTS

A escolha da linguagem C para o desenvolvimento do simulador esteve relacionada com a sua eficiência e familiarização da mesma, que permitem um controlo direto sobre a memória e a comunicação via sockets. Este fator revelou-se essencial para implementar de forma simples e eficaz a função Read Holding Registers (03), sem recorrer a bibliotecas externas. Para além disso, o C é uma linguagem consolidada em contexto académico e industrial, assegurando maior facilidade de uso.

Todavia, para a execução dos cenários de teste foi adotado o VBScript (.vbs). Esta linguagem, apesar de simples, revelou-se adequada ao objetivo de automatizar a alteração e leitura de registos, uma vez que possibilita a criação de instruções condicionais (if/else) e a interação direta com o simulador. O facto de poder ser escrita em qualquer editor de texto, como o Notepad, permitiram uma rápida criação e edição dos ficheiros necessários.

A combinação destas duas linguagens mostrou-se vantajosa: enquanto o C assegura a robustez do núcleo do simulador, o VBScript oferece flexibilidade na simulação de diferentes condições operacionais. Esta abordagem garantiu a preparação de cenários realistas de teste, fundamentais para a validação da plataforma eHub.

```
#  produção > 5 e SOC < 70 → HR400003 = 5

SET 4 400003 0
IFGT 4 400001 5 THEN SET 4 400003 5
IFGE 4 400002 70 THEN SET 4 400003 0
```

Figura 10- Exemplo VBScript

3.3. BASE DE DADOS

A plataforma eHub já se encontrava suportada por uma base de dados em MySQL, utilizada para armazenar e organizar as variáveis recolhidas dos dispositivos da microrrede. Este sistema foi definido no desenvolvimento original da aplicação e não sofreu alterações no âmbito deste estágio.

Para efeitos de exploração e validação dos dados, foi utilizada a ferramenta DBeaver, que permitiu visualizar as tabelas e efetuar consultas. A utilização desta interface gráfica simplificou o processo de verificação, evitando a necessidade de utilização da linha de comandos SQL.

De forma a garantir a comunicação entre o simulador, a base de dados e as duas máquinas virtuais (frontend e backend), foi necessário ajustar a configuração de rede e atualizar os endereços IP. Com esta adaptação, os registos simulados puderam ser corretamente interpretados pela aplicação e armazenados no sistema.

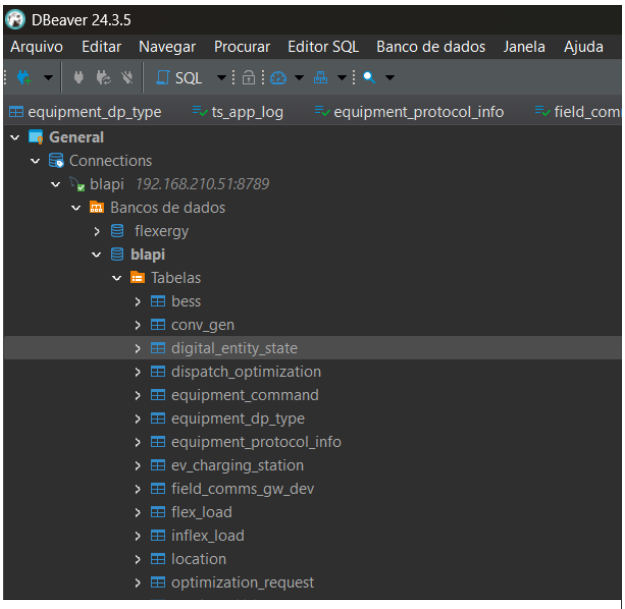


Figura 11- Base de dados MYSQL no Dbeaver

4. DESENVOLVIMENTO

4.1. CONSTRUÇÃO DO SIMULADOR

O desenvolvimento do simulador surgiu da necessidade de ter uma ferramenta capaz de reproduzir o comportamento de dispositivos compatíveis com o protocolo Modbus TCP, permitindo testar a plataforma eHub em condições próximas da realidade. Apesar da existência de simuladores como o ModRSsim2, optou-se por construir uma aplicação própria, de forma a garantir maior flexibilidade e enquadramento com os objetivos do estágio.

O simulador foi concebido para responder a três requisitos principais:

- Edição manual de registos – possibilidade de alterar e ver, através do terminal, os valores de registos do tipo Holding Registers.

- Capacidade de interpretar scripts automáticos – integração de ficheiros em VBScript (.vbs) que permitem simular cenários de teste definidos previamente.

- Compatibilidade com o eHub – assegurar a comunicação através da função Read Holding Registers (03), garantindo a troca de informação com a base de dados MySQL associada à plataforma.

Na figura 6 está presente a arquitetura do simulador Modbus TCP/IP, esta foi organizada de forma a representar a sua função como servidor, capaz de responder a pedidos de clientes externos (eHub e Modbustcptool). Internamente, a aplicação suporta tanto a edição manual de registos como a execução de scripts em VBScript, sendo ambos direcionados para os Holding Registers.

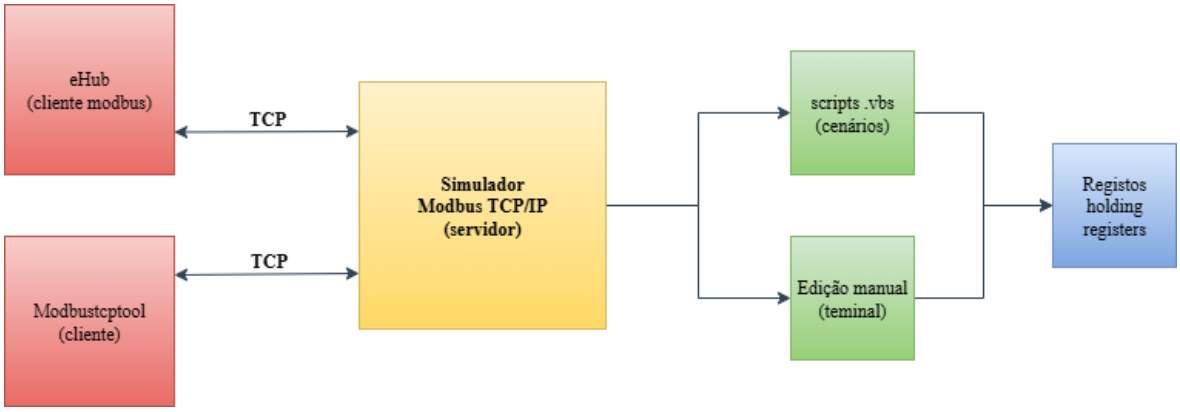


Figura 12- Arquitetura conceptual do simulador Modbus TCP/IP

4.2. GESTÃO DOS REGISTOS MODBUS

O módulo modbus_utils.c implementa a lógica associada à função 0x03 – Read Holding Registers, a única atualmente suportada pelo simulador. Os valores são armazenados num vetor global (holdingregister[65535]), que representa o espaço completo de Holding Registers.

A conversão entre endereços lógicos (ex.: HR400001) e índices internos é realizada pela função auxiliar hr_to_index(). Além disso, são disponibilizadas funções de Get e Set, utilizadas tanto pela consola como pelo motor de scripts, assegurando consistência no acesso. Sempre que ocorre alteração de valor, é feito logging automático para registo da mudança.

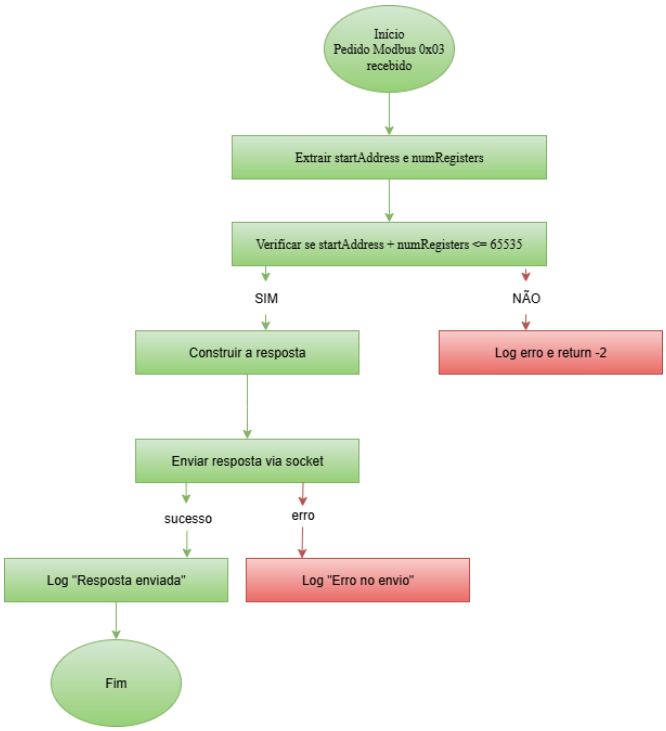


Figura 13- Fluxograma modbus_utils.c

4.3. GESTÃO DE SOCKETS

O módulo server_utils.c implementa a camada de comunicação TCP utilizada pelo simulador. Através deste módulo é possível:

- Criar o servidor (socket → bind → listen) na porta Modbus/TCP (502).
- Aceitar ligações de clientes (accept), retornando um socket dedicado por sessão.
- Ler dados de forma robusta (recv), com tratamento de erros e deteção do encerramento remoto.
- Verificar disponibilidade de dados (bCheckForData), que encapsula a função select() com timeout, permitindo esperar de forma não bloqueante.
- Limpar o socket (flushSocket) para descartar dados residuais antes de iniciar uma nova comunicação. Esta camada garante que o simulador pode atender múltiplas ligações sequenciais de forma fiável e tolerante a erros de rede.

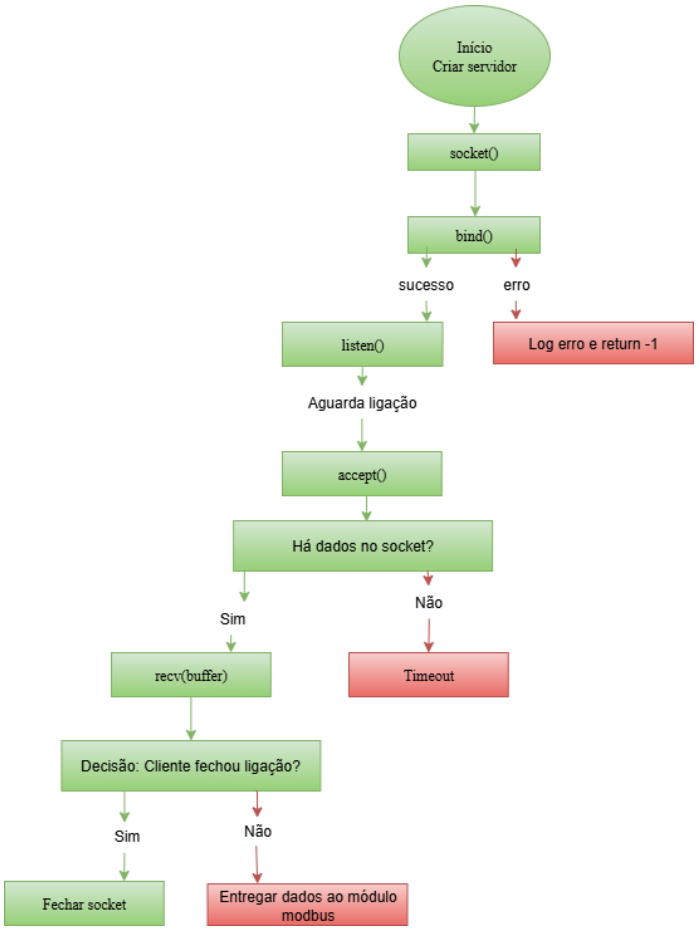


Figura 14- Fluxograma server_utils.c

4.4. CONSOLA DE INTERAÇÃO

O módulo consola.c implementa uma interface textual que permite ao utilizador interagir diretamente com o simulador antes da execução do servidor Modbus. Esta consola foi concebida como uma ferramenta de teste e depuração, dispensando temporariamente clientes externos e permitindo validar rapidamente o comportamento do sistema.

Através deste menu, o utilizador pode escolher entre quatro opções principais: mudar o valor de um registo (SET), ler o valor de um registo (GET), iniciar o servidor Modbus ou carregar e executar um script de teste. No primeiro caso, o programa solicita o endereço do registo (ex.: HR400001) e o novo valor, atualizando a posição correspondente no vetor de Holding Registers através da função SetRegisterValue(). De forma análoga, na opção de leitura é solicitado o endereço e devolvido o valor associado, recorrendo a GetRegisterValue().

A opção de iniciar o servidor termina o ciclo da consola e transfere o controlo para a rotina principal que gere as comunicações TCP. Por fim, a execução de scripts é realizada a partir de um caminho indicado pelo utilizador, sendo o ficheiro interpretado pelo motor definido em run_vbs.c.

O funcionamento desta rotina pode ser representado pelo fluxograma da Figura 4.4, onde se observa o ciclo de repetição do menu e os diferentes ramos de execução de acordo com a escolha do utilizador.

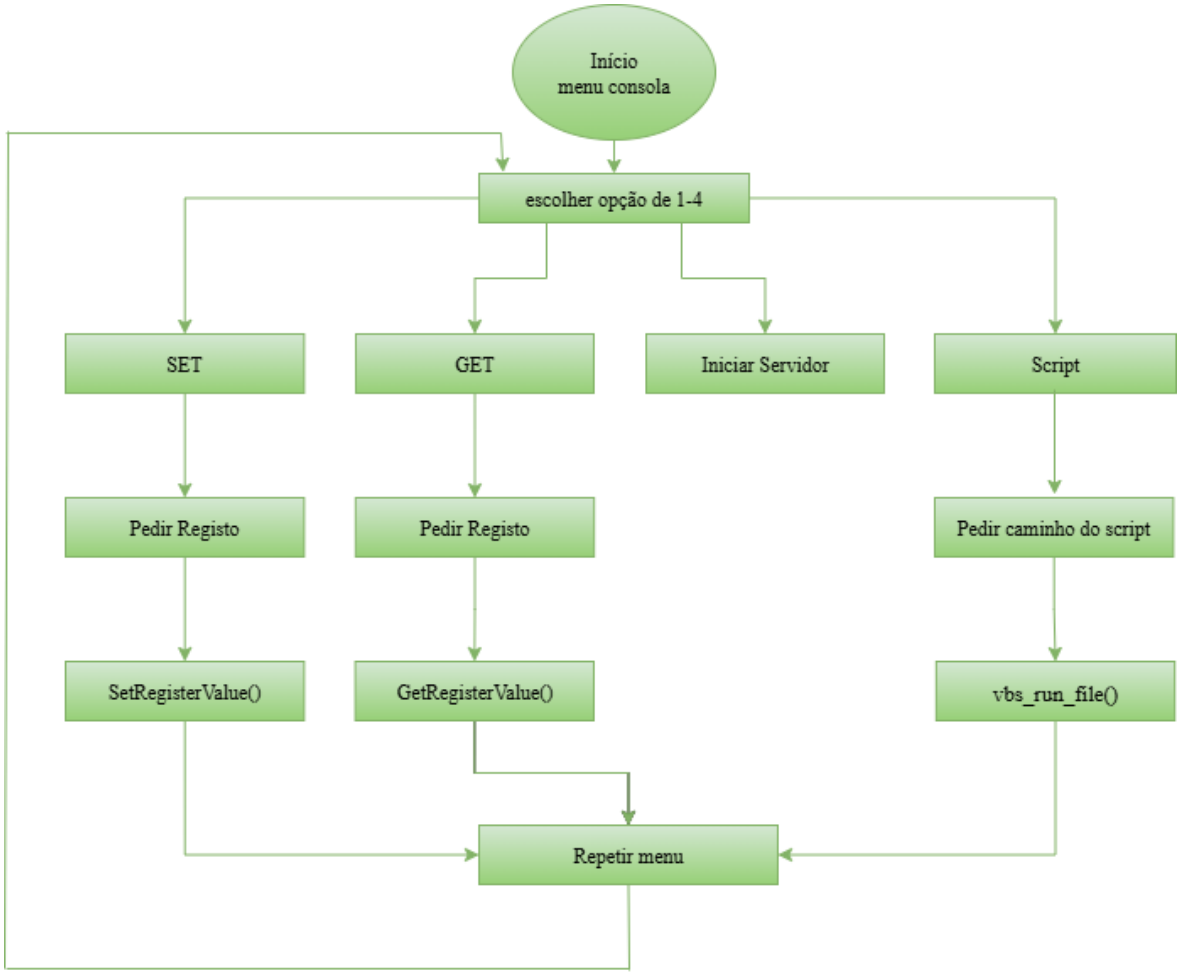


Figura 15- Fluxograma consola.c

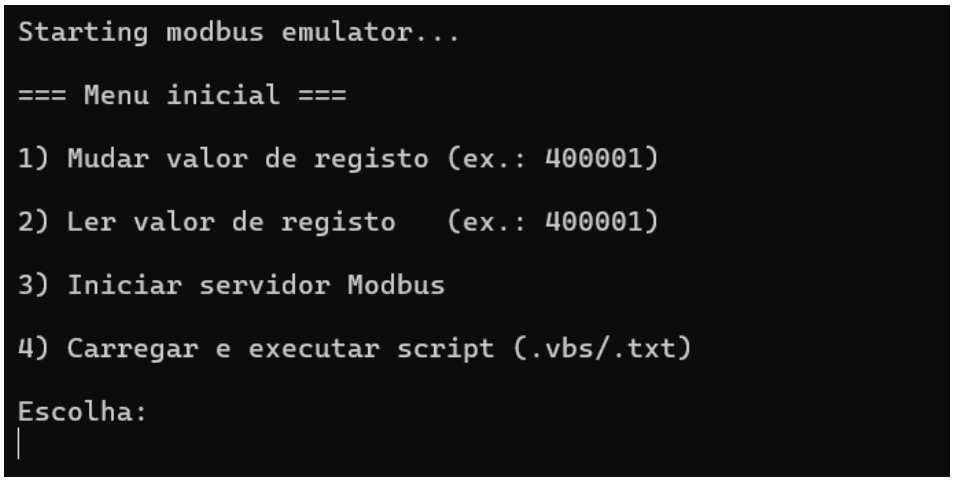


Figura 16- Consola do Simulador

4.5. MOTOR DE SCRIPTS

O módulo run_vbs.c constitui o motor de execução de scripts, permitindo reproduzir cenários de teste através de ficheiros de texto simples com comandos estruturados. Cada linha de um ficheiro de script é lida sequencialmente e posteriormente interpretada. O comando identificado é convertido para maiúsculas e analisado, de modo a decidir a ação a executar.

Os comandos disponíveis dividem-se em três grupos principais. O primeiro é o SET, que permite definir o valor de um registo específico através de SetRegisterValue(). O segundo é o GET, que lê o valor de um registo e o apresenta no terminal recorrendo a GetRegisterValue(). O terceiro grupo corresponde às instruções condicionais, representadas por variantes como IFGT (If Greater Than (>)), IFLT (If Less Than (<)), IFEQ (If Equal (=)), IFGE (If Greater or Equal (≥)), IFLE (If Less or Equal (≤)) e IFNE (If Not Equal (≠)). Estas instruções comparam o valor de um registo com uma constante e, caso a condição seja verdadeira, fazem com que seja atribuído um novo valor a outro registo.

Desta forma, o motor de scripts permite reproduzir situações reais de operação, como o carregamento e descarga de baterias, deteção de défices energéticos ou ativação de alarmes de consumo.

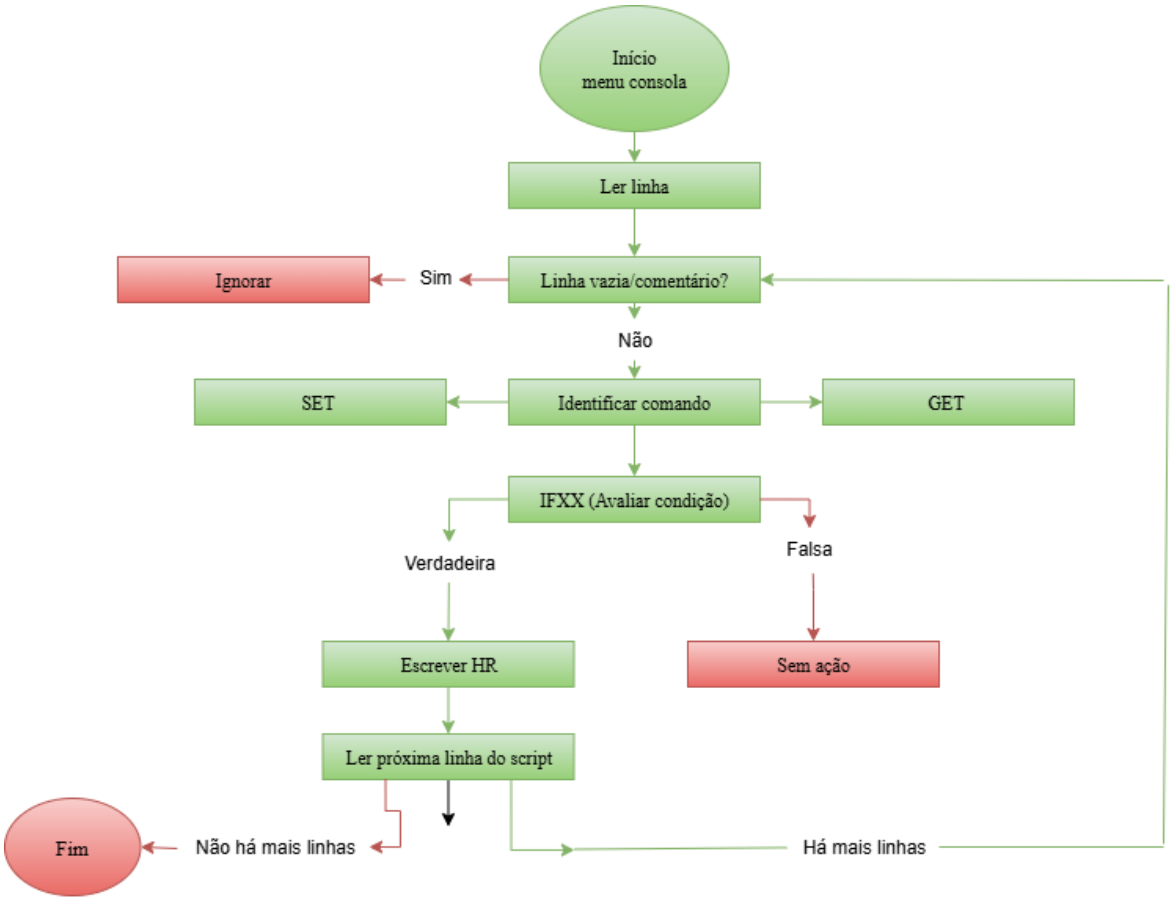


Figura 17- Fluxograma run_vbs.c

4.6. FUNÇÃO PRINCIPAL

O ficheiro main.c constitui o núcleo do simulador, assegurando a coordenação entre todos os módulos desenvolvidos. Em primeiro lugar, o utilizador interage com o programa através da consola, onde pode definir registos ou carregar scripts de teste. Após esta etapa inicial invoca as rotinas responsáveis pela criação do servidor TCP, estabelecendo o porto e preparando o sistema para receber clientes externos.

A função principal é responsável por garantir que novas ligações sejam aceites, que os pedidos válidos sejam encaminhados para o módulo Modbus, e que as sessões sejam encerradas quando ocorre falha ou desconexão. Esta lógica centraliza o funcionamento do simulador, permitindo que os restantes módulos — gestão de registos, comunicação de rede, consola e motor de scripts — funcionem de forma coerente.

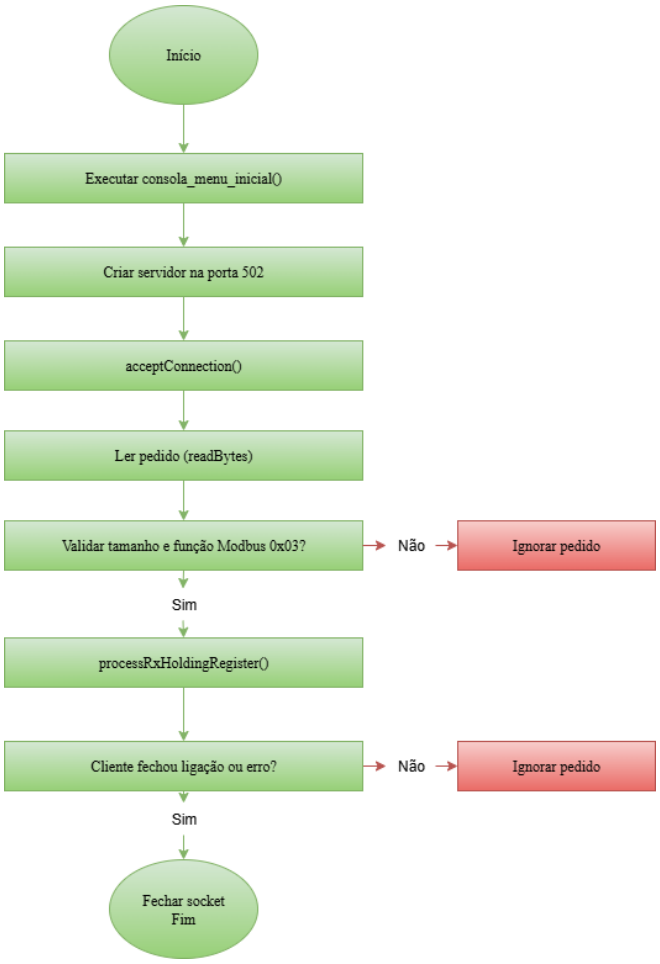


Figura 18- Fluxograma main.c

5. RESULTADOS

5.1. ESTADO ATUAL DA INFRAESTRUTURA

No final do estágio, foi concluída a preparação da infraestrutura necessária para a realização dos ensaios à plataforma eHub. Foram configuradas duas máquinas virtuais dedicadas, correspondentes ao frontend e backend do sistema, com os respetivos backups restaurados e a base de dados MySQL operacional e acessível através da ferramenta DBeaver.

O simulador Modbus, desenvolvido em linguagem C, encontra-se totalmente funcional, suportando a função 0x03 – Read Holding Registers e permitindo tanto a interação manual, via consola, como a execução automatizada de scripts em VBScript (.vbs). Esta ferramenta constitui o núcleo do trabalho realizado, pois garante a simulação de dispositivos industriais necessários para validar os cenários da microrrede.

Adicionalmente, foram elaborados 30 cenários de teste, cada um com um guião de execução associado, capazes de reproduzir condições reais como variações de consumo, falhas de comunicação ou ciclos de carga e descarga de baterias. Embora os testes ainda não tenham sido realizados sobre a plataforma eHub, a infraestrutura encontra-se pronta e validada, cumprindo todos os requisitos necessários para a sua execução.

5.2. CENÁRIOS DE TESTE DEFINIDOS

Durante o estágio foram realizados 30 cenários de simulação, cada um associado a um script em VBScript (.vbs), que permitem reproduzir condições reais de funcionamento da microrrede. Estes cenários contem desde operações normais, como ciclos de carga e descarga de baterias, até situações de falha, incluindo défices energéticos, alarmes de comunicação e comportamentos anormais de consumo.

O documento que descreve estes cenários foi submetido à verificação e validação pelo cliente (EFACEC), garantindo que as situações contempladas se encontram alinhadas com os requisitos funcionais da plataforma eHub e com os objetivos do projeto Asprela+ Sustentável.

Devido ao seu número e detalhe, os cenários não são apresentados individualmente neste relatório. No entanto, a Tabela 5.1 apresenta alguns exemplos representativos, que ilustram a lógica seguida na sua definição.

Tabela 4- Exemplos de cenários de teste definidos

Nº do Cenário	Descrição	Objetivo	Resultado Esperado
1	Ciclo normal de carga e descarga de baterias	Avaliar o comportamento da plataforma em condições de operação normal	Valores de carga/descarga registados corretamente e refletidos no eHub
7	Défice energético na microrrede	Testar a capacidade de detetar e sinalizar insuficiência de produção	Geração de alarme de défice e registo no sistema
14	Falha de comunicação com um dispositivo Modbus	Verificar a robustez da plataforma perante perda de ligação	Emissão de alarme de comunicação e manutenção dos últimos valores válidos
25	Sobrecarga de consumo em períodos de pico	Avaliar a resposta da plataforma a um aumento súbito da procura	Registo da sobrecarga, emissão de alarme e eventual ativação de medidas corretivas

5.3. CENÁRIOS DE TESTE DEFINIDOS

De forma a demonstrar o correto funcionamento do simulador desenvolvido, foram recolhidos alguns exemplos práticos da sua utilização. Estes exemplos permitem evidenciar tanto a interação manual via consola como a execução de cenários automatizados através de scripts.

Na Figura 19 é apresentado um exemplo de alteração manual de um registo (função SET), onde o utilizador define um novo valor para um endereço específico. De seguida, foi utilizada a opção 2 (GET) para ler o mesmo registo, confirmando que o valor se manteve armazenado corretamente. Este procedimento permite validar de forma simples a integridade das operações de escrita e leitura sobre os *holding registers*.

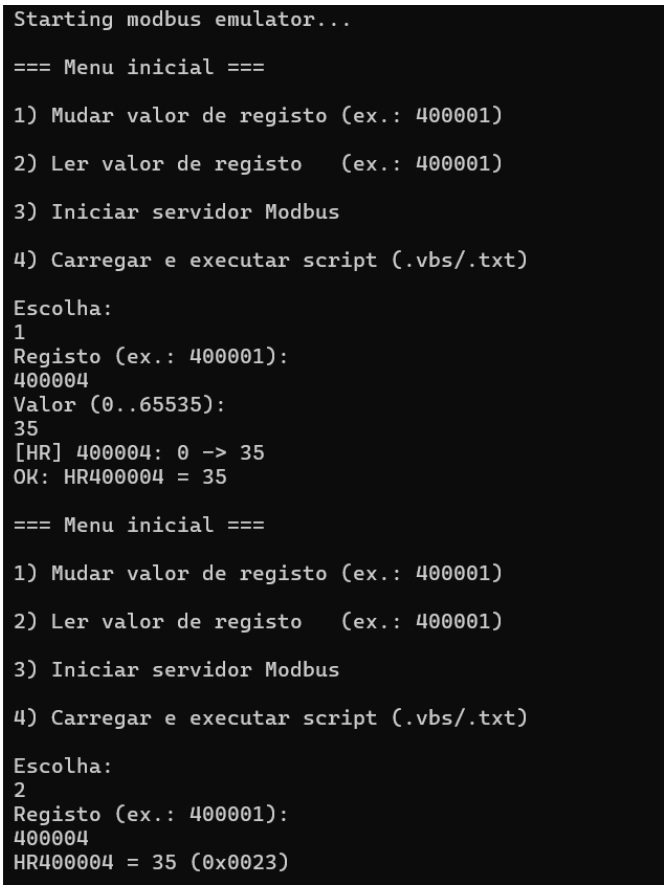


Figura 19- Exemplo de alteração manual de um registo (SET) e verificação posterior através de leitura (GET) no simulador Modbus.

Na Figura 5.3 é apresentado um exemplo de execução de um script em VBScript no simulador Modbus. Antes da execução do guião, o registo HR400001 (produção) foi configurado manualmente com o valor 6 e o registo HR400002 (estado de carga da bateria – SOC) com o valor 35. O script utilizado define inicialmente o registo HR400003 como 0, verificando depois se a produção (HR400001) é superior a 5. Como a condição é verdadeira e o SOC está abaixo de 70, o simulador altera automaticamente o valor de HR400003 para 5, confirmando a atualização em tempo real através da consola.

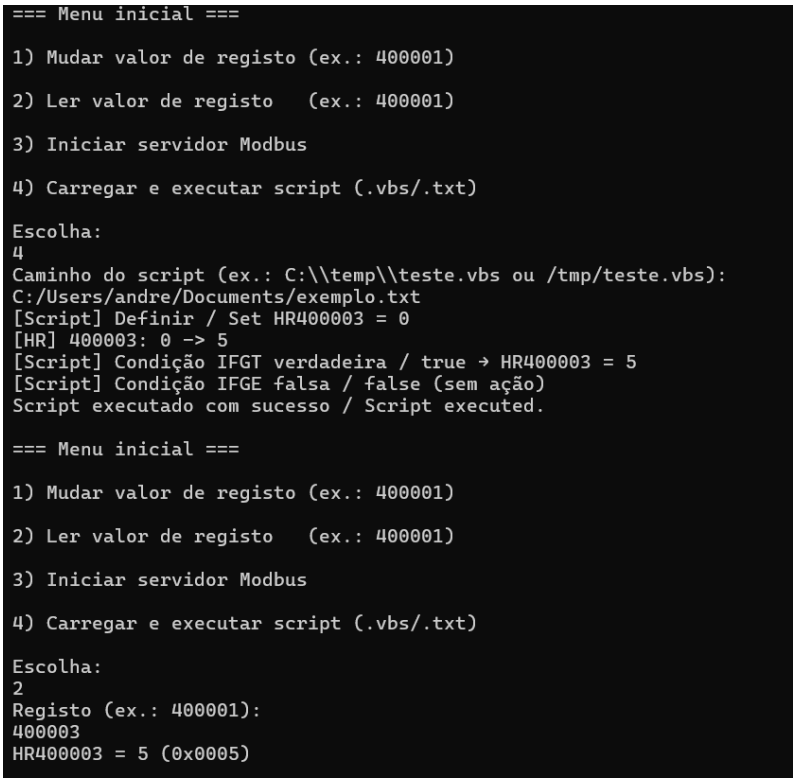


Figura 20- Execução de script em VBScript no simulador

5.4. AVALIAÇÃO QUALITATIVA E RESULTADOS ESPERADOS

Apesar de os testes completos sobre a plataforma eHub não terem ainda sido realizados, foi possível definir critérios de avaliação e realizar ensaios preliminares diretamente sobre o simulador. Estes ensaios confirmaram que a ferramenta responde de forma estável e coerente, constituindo uma base sólida para a fase de integração.

Em termos qualitativos, o simulador demonstrou:

- Estabilidade em funcionamento contínuo, sem falhas de comunicação ou encerramentos inesperados.
- Robustez perante pedidos inválidos, devolvendo mensagens de erro adequadas.
- Consistência nos registos, mantendo corretamente os valores definidos em operações de escrita.
- Compatibilidade com clientes Modbus TCP externos (ex.: ModbusTCPtool), assegurando conformidade com o protocolo.

Do ponto de vista quantitativo, foram definidos valores de referência a cumprir durante a fase de testes:

- Consistência de leituras: 100% de correspondência entre leituras consecutivas do mesmo registo.
- Deteção de pedidos inválidos: 100% de rejeição quando os endereços se encontram fora do intervalo válido.
- Execução de scripts: interpretação correta de todas as instruções (SET, GET e IF).

6. CONCLUSÕES

6.1. DISCUSSÃO E CONCLUSÃO

O trabalho desenvolvido durante o estágio conseguiu atingir o principal objetivo definido: criar as condições necessárias para a realização de testes à plataforma eHub no contexto do projeto Asprela+ Sustentável. O contributo mais relevante foi o desenvolvimento de um simulador Modbus TCP/IP em linguagem C, capaz de suportar a função 0x03 – Read Holding Registers, permitindo tanto a interação manual via consola como a execução automatizada de cenários através de scripts em VBScript (.vbs).

Com este simulador, foi possível preparar uma infraestrutura completa de testes, composta por máquinas virtuais para o frontend e backend do eHub, bem como pela base de dados MySQL devidamente configurada. Para além disso, foram definidos 30 cenários de teste, validados pelo cliente (EFACEC), cobrindo tanto situações normais de operação como falhas de comunicação ou défices energéticos.

Assim, pode-se afirmar que o estágio alcançou os objetivos propostos, contribuindo para o projeto não apenas com uma ferramenta funcional e reutilizável, mas também com uma metodologia de teste organizada e alinhada com os requisitos da comunidade energética em estudo.

6.2. FATORES LIMITADORES

Apesar dos resultados obtidos terem sido positivos, alguns fatores limitaram a extensão do trabalho:

- Cobertura funcional restrita no simulador, já que apenas foi implementada a função 0x03 (Read Holding Registers). Outras funções Modbus (escrita de registos, leitura de entradas digitais, etc.) não foram contempladas devido à plataforma eHub apenas trabalhar com a função 0x03

- Testes finais por realizar: embora a infraestrutura e os cenários estejam completos, a execução integral dos testes sobre a plataforma eHub ainda não foi concretizada, pelo que a avaliação quantitativa direta do desempenho do sistema ficou em aberto.

- Interface limitada: a interação faz-se exclusivamente por consola de texto, o que, apesar de suficiente para a fase de desenvolvimento, não é a abordagem mais prática para utilizadores sem experiência técnica.

6.3. MELHORIAS PARA TRABALHO FUTURO

Como continuidade natural do trabalho desenvolvido, as melhorias futuras poderão incidir sobretudo na automação da ferramenta. Embora o simulador tenha sido implementado apenas com suporte à função 0x03 (Read Holding Registers), esta opção deveu-se ao facto de a plataforma eHub comunicar exclusivamente através de registos do tipo holding registers, pelo que não se justificava o desenvolvimento de funcionalidades adicionais fora do âmbito do projeto.

Deste modo, uma evolução relevante passaria pela criação de uma interface gráfica que substituísse ou complementasse a atual interação por consola de texto. Essa interface permitiria configurar registos, carregar scripts e acompanhar resultados de forma mais intuitiva, facilitando o uso por parte de técnicos com menos experiência em programação.

Outra possibilidade de melhoria seria a automatização da análise de resultados. Atualmente, a verificação é feita manualmente, mas no futuro o simulador poderá incluir mecanismos de geração automática de relatórios, com métricas quantitativas e representações gráficas que documentem o desempenho de cada cenário de teste.

Por fim, destaca-se o potencial de integração contínua com a plataforma eHub, possibilitando que os cenários de teste sejam executados de forma periódica e automática. Esta evolução permitiria que o simulador funcionasse também como uma ferramenta de monitorização, contribuindo para a deteção precoce de anomalias e para a operação estável e sustentável da microrrede.

Referências Documentais

[1] MODBUS ORGANIZATION, MODBUS PROTOCOL OVERVIEW, DISPONÍVEL EM: HTTPS://MODBUS.ORG, (ACESSO EM SETEMBRO 2025)

[2] MODBUS ORGANIZATION, MODBUS APPLICATION PROTOCOL SPECIFICATION V1.1B3, 2012.

[3] EFACEC, ASPRELA+ SUSTENTÁVEL – MEMÓRIA DESCRITIVA, 2025.

[4] EFACEC, ASPRELA+ SUSTENTÁVEL – PLANO DE ENSAIOS DO SISTEMA DE GESTÃO DA MICRORREDE, 2025.

[5] TEKON ELECTRONICS, ESTRUTURA MODBUS TCP, DISPONÍVEL EM: HTTPS://WWW.TEKONELECTRONICS.COM/IMAGEM/BLOG/ESTRUTURA%20MODBUS%20TCP%20PT.PNG, (ACESSO EM SETEMBRO 2025).

[6] TEKON ELECTRONICS, DIAGRAMA DOS DADOS MODBUS TCP/IP, DISPONÍVEL EM: HTTPS://WWW.TEKONELECTRONICS.COM/IMAGEM/BLOG/DIAGRAMA%20DADOS%20MDOBUS%20TCP%20PT.PNG, (ACESSO EM SETEMBRO 2025).

[7] T. SAUTER, “THE THREE GENERATIONS OF FIELD-LEVEL NETWORKS—EVOLUTION AND COMPATIBILITY ISSUES,” IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS, VOL. 57, NO. 11, PP. 3585–3595, 2010.

[8] K. SCHWIEGER, PRACTICAL MODBUS TESTING WITH SIMULATION TOOLS, IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS, 2019.

[9] MODRSSIM2, MODBUS SIMULATOR TOOL, DISPONÍVEL EM: HTTP://WWW.MODRSSIM2.COM, (ACESSO EM SETEMBRO 2025).

[10] R. H. LASSETER, “MICROGRIDS,” IEEE POWER ENGINEERING SOCIETY WINTER MEETING, VOL. 1, PP. 305–308, 2002.

[11] EFACEC, ASPRELA+ SUSTENTÁVEL – PLANO DE ENSAIOS DO SISTEMA DE GESTÃO DA MICRORREDE, 2025.

Anexo A. Cenários de simulação

A.1 Carregamento por produção elevada e SOC baixo

Se a produção instantânea (HR400001) for superior a 5 kW e o SOC (HR400002) da bateria for inferior a 70%, ativar comando de carregamento (HR400003).

```
# Cenário: Produção > 5 kW e SOC < 70% → HR400050 = 5

SET 4 400050 0
IFGT 4 400041 5 THEN SET 4 400050 5
IFGE 4 400042 70 THEN SET 4 400050 0
```

Listagem A. 1- Script em VBScript para carregamento por produção elevada e SOC baixo.

A.2 Preparação energética por atividade agendada

Se existir uma atividade agendada e a produção atual for superior a 2 kW, autorizar o abastecimento/preparação energética.

```
# Cenário: Agendamento ativo (=1) E produção > 2 kW → HR400150 = 14

SET 4 400150 0
IFEQ 4 400141 1 THEN SET 4 400150 14
IFLE 4 400142 2 THEN SET 4 400150 0
```

Listagem A. 2- Script em VBScript para preparação energética por atividade agendada

A.3 Script de produção zero em horário solar

Sinalizar possível anomalia quando, em horário normalmente com radiação (10h–17h), a produção da UPAC é zero.

```
# Cenário: 10 ≤ Hora ≤ 17 e Produção == 0 → HR400160 = 16

SET 4 400160 0
IFEQ 4 400151 0 THEN SET 4 400160 16
IFLT 4 400152 10 THEN SET 4 400160 0
IFGT 4 400152 17 THEN SET 4 400160 0
```

Listagem A. 3- Script em VBScript para Produção zero em horário solar

Anexo B. Código-fonte do simulador

B.1 main.c

```
#include <stdio.h>
#include <stdlib.h>
#include "user_interface.h"
#include "server_utils.h"
#include "modbus_utils.h"
#include "run_vbs.h"
#include "consola.h"
#include <sys/socket.h>
#include <cygwin/in.h>
#include <strings.h>
#define DEFAULT_PORT 502
#define READ_HOLDING_REGISTER 0x03

int main(int argc, char** argv)
{
    Print("Starting modbus emulator...");
    consola_menu_inicial();
    int so_server = createServer(DEFAULT_PORT);

    if(so_server < 0)
    {
        return -1;
    }
    while(1)
    {
        int so_client = acceptConnetion(so_server);


        if(so_client < 0)
        {
            break;
        }
        while (1)
        {
            unsigned char buffer[260];
            int readSize = 0;
            int r = readBytes(so_client, (char*)buffer, sizeof(buffer),
&readSize);

            if (r == -2)
            {
                close(so_client);
                break;
            }
            if (r < 0)
            {
                close(so_client);
                break;
            }
            if (r == 0)
            {
                usleep(10 * 1000);
                continue;
            }

            if (readSize < 12) continue;
            if (buffer[7] != 0x03) continue;
            processRxHoldingRegister(so_client, buffer, readSize);
        }
    }
    return (EXIT_SUCCESS);
```

```
}
```

Listagem A. 4- main.c

B.2 modbus_utils.c

```
#include "user_interface.h"
#include "server_utils.h"
#include <stdio.h>
#include <sys/socket.h>
#include <string.h>
unsigned short holdingregister[65535];
int processRxHoldingRegister(int so_client, const unsigned char* buffer, int
bufferSize)
{
    uint16_t startAddress = ((unsigned char)buffer[8]  << 8) | (unsigned
char)buffer[9];
    uint16_t numRegisters = ((unsigned char)buffer[10] << 8) | (unsigned
char)buffer[11];
    Print("Pedido: start=%u qty=%u  HR[0]=%u  HR[start]=%u",startAddress,
numRegisters,(unsigned)holdingregister[0],(unsigned)holdingregister[startAddress]
);
    if (startAddress + numRegisters > 65535)
    {
        Print("Error...");
        return -2;
    }
    unsigned char response[256];
    response[0] = buffer[0];
    response[1] = buffer[1];
    response[2] = buffer[2];
    response[3] = buffer[3];
    unsigned short pdu_len = 1 /*unit*/ + 1 /*func*/ + 1 /*byte count*/ +
(numRegisters * 2);
    response[4] = (pdu_len >> 8) & 0xFF;
    response[5] =  pdu_len       & 0xFF;
    response[6] = buffer[6];
    response[7] = 0x03;
    response[8] = numRegisters * 2;
    for (int i = 0; i < numRegisters; i++)
    {
       unsigned short v = holdingregister[startAddress + i];
            response[9 + i * 2]     = (v >> 8) & 0xFF;
            response[9 + i * 2 + 1] = v & 0xFF;
    }
    int responseSize = 9 + numRegisters * 2;
    Print("A enviar resposta para o cliente:");

      for (int i = 0; i < responseSize; i++)
      {
        printf("%02X ", (unsigned char)response[i]);
      }
    ssize_t result = send(so_client, response, responseSize, 0);
    if (result < 0)
    {
        Print("Error sending response...");
        return -1;
    }
    Print("Response sent to the client.");
    return 0;
}
static int hr_to_index(unsigned int hr)
{
    if (hr >= 400001) return (int)(hr - 400001);
    if (hr >= 1 && hr <= 65535) return (int)(hr - 1);
```

```
    return (int)hr;
}
unsigned short GetRegisterValue(int unit, unsigned int hr)
{
    (void)unit;
    int idx = hr_to_index(hr);
    if (idx < 0 || idx >= 65535) return 0;
    return holdingregister[idx];
}
void SetRegisterValue(int unit, unsigned int hr, unsigned short value)
{
    (void)unit;
    int idx = hr_to_index(hr);
    if (idx < 0 || idx >= 65535) return;
    unsigned short old = holdingregister[idx];
    holdingregister[idx] = value;
    if (old != value)
    {
        printf("[HR] %06u: %u -> %u\n", (unsigned)(400001 + idx), old, value);
        fflush(stdout);
    }
}
```

Listagem A. 5- modbus_utils.c

B.3 Server_utils.c

```
#include <sys/socket.h>
#include <cygwin/in.h>
#include <string.h>
#include "user_interface.h"
#include "server_utils.h"
int createServer(int port)
{
    struct sockaddr_in serv_addr, cli_addr;
    int so_server = socket(AF_INET, SOCK_STREAM, 0);
    bzero(&serv_addr,sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_addr.s_addr = INADDR_ANY;
    serv_addr.sin_port = htons(port);
    if(bind(so_server, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0)
    {
        Print("Error! Unable to bind...");
        return -1;
    }
    if(listen(so_server,5) < 0)
    {
        Print("Error! Unable to listen...");
        return -2;
    }
    return so_server;
}
int acceptConnetion(int so_server)
{
    int so_client = accept(so_server, NULL, NULL);
    if (so_client < 0)
    {
        Print("Error! Unable to accept...");
        return -3;
    }
    Print("Ligação aceite... socket: %d", so_client);
    return so_client;
}
int readBytes(int so_client, char *buffer, int maxsize, int *receivesize)
{
```

```
    if (!bCheckForData(1000, so_client)) {
        return 0;
    }
    int n = recv(so_client, buffer, maxsize, 0);
    if (n < 0) { perror("recv"); return -1; }
    if (n == 0) { Print("Cliente fechou a ligação."); return -2; }
    if (receivesize) *receivesize = n;
    return n;
}
char bCheckForData(uint32_t dwTimeout, int Socket)
{
    fd_set readfds;
    struct timeval time;
    int r;
    if (Socket < 0)
        return 0;
    FD_ZERO(&readfds);
    FD_SET(Socket, &readfds);
    time.tv_sec  = dwTimeout / 1000;
    time.tv_usec = (dwTimeout % 1000) * 1000;
    r = select(Socket + 1, &readfds, NULL, NULL, &time);
    if (r < 0)
    {
        Print("select() falhou.");
        return 0;
    }
    if (r == 0) return 0;
    return FD_ISSET(Socket, &readfds) ? 1 : 0;
}
void flushSocket(int sock)
{
    char dump[128];
    while (bCheckForData(100, sock))
    {
        recv(sock, dump, sizeof(dump), 0);  // lê e descarta
    }
    Print("Socket limpo.");
}
```

Listagem A. 6- server_utils.c

B.4 Consola.c

```
#include "user_interface.h"
#include "modbus_utils.h"
#include "server_utils.h"
#include "run_vbs.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/select.h>
#include <sys/types.h>
#include <unistd.h>
void consola_menu_inicial(void) {
    char line[64];
    while (1) {
        Print("\n=== Menu inicial ===\n");
        Print("1) Mudar valor de registo (ex.: 400001)\n");
        Print("2) Ler valor de registo   (ex.: 400001)\n");
        Print("3) Iniciar servidor Modbus\n");
        Print("4) Carregar e executar script (.vbs/.txt)\n");
        Print("Escolha: ");
        fflush(stdout);
```

```
        if (!fgets(line, sizeof(line), stdin)) {
            break;
        }
        int opcao = atoi(line);
        if (opcao == 1)
        {
            unsigned hr = 0, val = 0;
            Print("Registo (ex.: 400001): ");
            fflush(stdout);
            if (!fgets(line, sizeof(line), stdin)) continue;
            if (sscanf(line, "%u", &hr) != 1) { Print("Endereço inválido /
Invalid address."); continue; }
            Print("Valor (0..65535): ");
            fflush(stdout);
            if (!fgets(line, sizeof(line), stdin)) continue;
            if (sscanf(line, "%u", &val) != 1 || val > 65535) { Print("Valor
inválido / Invalid value."); continue; }
            SetRegisterValue(1, hr, (unsigned short)val);
            Print("OK: HR%06u = %u", hr, val);
        }
        else if (opcao == 2)
        {
            unsigned hr = 0;
            Print("Registo (ex.: 400001): ");
            fflush(stdout);
            if (!fgets(line, sizeof(line), stdin)) continue;
            if (sscanf(line, "%u", &hr) != 1) { Print("Endereço inválido /
Invalid address."); continue; }
            unsigned short v = GetRegisterValue(1, hr);
            Print("HR%06u = %u (0x%04X)", hr, (unsigned)v, (unsigned)v);
        }
        else if (opcao == 4)
        {
            char path[256];
            Print("Caminho do script (ex.: C:\\\\temp\\\\teste.vbs ou
/tmp/teste.vbs): ");
            fflush(stdout);
            if (!fgets(path, sizeof(path), stdin))
            {
                Print("Entrada inválida / Invalid input.");
                continue;
            }
            size_t n = strlen(path);
            while (n && (path[n-1] == '\n' || path[n-1] == '\r')) path[--n] = 0;
            int rc = vbs_run_file(path);
            if (rc == 0)  { Print("Script executado com sucesso / Script
executed."); }
            else          { Print("Falha ao executar script / Failed to execute
script."); }
        }
        else if (opcao == 3)
        {
            break;
        }
        else
        {
            Print("Opção inválida / Invalid option.");
        }
    }
}
```

Listagem A. 7- consola.c

B.5 run_vbs.c

```
#include "user_interface.h"
#include "server_utils.h"
#include "consola.h"
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

static void rstrip(char *s){
    size_t n = strlen(s);
    while (n && (s[n-1]=='\r' || s[n-1]=='\n' || s[n-1]==' ' || s[n-1]=='\t'))
s[--n]=0;
}
static void lskip_spaces(const char **ps){
    while (**ps==' ' || **ps=='\t') (*ps)++;
}
static int is_int(const char *s){
    if (!s || !*s) return 0;
    if (*s=='+'||*s=='-') s++;
    if (!*s) return 0;
    for (; *s; ++s) if (*s<'0'||*s>'9') return 0;
    return 1;
}
static int read_i3(const char *pos, int *a,int *b,int *c){
    return (sscanf(pos, " %d %d %d", a,b,c)==3);
}
static int read_i2(const char *pos, int *a,int *b){
    return (sscanf(pos, " %d %d", a,b)==2);
}
static int read_if_then_set(const char *pos,int *cu,int *ca,int *cst,int *tu,int
*ta,int *tv)
{

    int n = sscanf(pos, " %d %d %d THEN SET %d %d %d", cu,ca,cst, tu,ta,tv);
    return (n==6);
}

int vbs_run_file(const char* path)
{
    FILE* f = fopen(path, "r");
    if (!f) {
        perror("open script");
        Print("Erro a abrir script / Failed to open script: %s", path);
        return -1;
    }
    char line[256];
    int lineno = 0;
    while (fgets(line, sizeof(line), f)) {
        lineno++;
        rstrip(line);
        const char *p = line;
        lskip_spaces(&p);
        if (*p==0 || *p=='#' || *p=='\'') continue;
        char cmd[16]={0};
        int i=0;
        while (p[i] && !isspace((unsigned char)p[i]) && i< (int)sizeof(cmd)-1) {
            cmd[i] = (char)toupper((unsigned char)p[i]);
            i++;
        }
        cmd[i]=0;
        const char *rest = p+i;
        lskip_spaces(&rest);
        if (strcmp(cmd,"SET")==0) {
            int u, hr, v;
            if (read_i3(rest, &u,&hr,&v)) {
                SetRegisterValue(u, hr, (unsigned short)v);
                Print("[Script] Definir / Set HR%06d = %d", hr, v);
```

```
            } else {
                Print("[Script] Linha inválida (SET) na linha %d: %s", lineno,
line);
            }
            continue;
        }
        if (strcmp(cmd,"GET")==0) {
            int u, hr;
            if (read_i2(rest, &u,&hr)) {
                unsigned short val = GetRegisterValue(u, hr);
                Print("[Script] Ler / Get HR%06d = %u", hr, val);
            } else {
                Print("[Script] Linha inválida (GET) na linha %d: %s", lineno,
line);
            }
            continue;
        }
        if (!strcmp(cmd,"IFGT") || !strcmp(cmd,"IFLT") || !strcmp(cmd,"IFEQ") ||
            !strcmp(cmd,"IFGE") || !strcmp(cmd,"IFLE") || !strcmp(cmd,"IFNE"))
        {
            int cu, ca, cst, tu, ta, tv;
            if (!read_if_then_set(rest, &cu,&ca,&cst, &tu,&ta,&tv)) {
                Print("[Script] Linha inválida (%s) na linha %d: %s", cmd,
lineno, line);
                continue;
            }
            unsigned short regVal = GetRegisterValue(cu, ca);
            int cond = 0;
            if (!strcmp(cmd,"IFGT")) cond = (regVal >  cst);
            if (!strcmp(cmd,"IFLT")) cond = (regVal <  cst);
            if (!strcmp(cmd,"IFEQ")) cond = (regVal == cst);
            if (!strcmp(cmd,"IFGE")) cond = (regVal >= cst);
            if (!strcmp(cmd,"IFLE")) cond = (regVal <= cst);
            if (!strcmp(cmd,"IFNE")) cond = (regVal != cst);

            if (cond) {
                SetRegisterValue(tu, ta, (unsigned short)tv);
                Print("[Script] Condição %s verdadeira / true → HR%06d = %d",
cmd, ta, tv);
            } else {
                Print("[Script] Condição %s falsa / false (sem ação)", cmd);
            }
            continue;
        }
        Print("[Script] Ignorado / Ignored (linha %d): %s", lineno, line);
    }
    fclose(f);
    return 0;
}
```

Listagem A. 8- run_vbs.c

B.6 user_interface.c

```
#include <stdarg.h>
#include <stdio.h>

void Print(const char * fmt, ...)
{
    char caBuffer[4096];
    va_list arglist;
    va_start(arglist, fmt);
    vsprintf(caBuffer, fmt, arglist);
    printf("%s\n", caBuffer);
```

```
    va_end(arglist);
}
```

Listagem A. 9- user_interface.c